i. Perkenalan

Milestone 3

Nama : Dianah Salsha Dilla

Batch : CODA-RMT-014

Program ini dibuat untuk melakukan evaluasi kemampuan penggunaan pipeline automation khususnya pada Phase 1

ii. Data Loading

Data yang digunakan berasal dari Kaggle dengan Judul Hajj and Umrah Crowd Management Dataset (2024) (referensi: https://www.kaggle.com/datasets/yarayahyahassan/hajj-and-umrah-crowd-management-dataset-2024)

```
#Menginstall library Great Expectations ke dalam sistem Google Colab
!pip install -q great-expectations
```

```
#a. Memanggil Libraries kedalam Notebook
#Mengolah dan memuat dataset dalam bentuk dataframe
import pandas as pd
#Mempersiapkan validasi data (7 expectations)
import great_expectations as ge
#Membuat konteks penyimpanan konfigurasi validasi
from great_expectations.data_context import FileDataContext
#Memastikan jalur file dataset sudah benar di sistem
import os
```

```
#b. Mengimport data kedalam Notebook
df = pd.read_csv('P2M3_Dianah_Salsha_Dilla_data_raw.csv')
df
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
```

	Timestamp	Location_Lat	Location_Long	Crowd_Density	Movement_Speed	Activity_Type	Weather_Conditions	Temperature	Sour
0	2024-02-28 05:36:50	21.258846	39.983949	Medium	0.60	Tawaf	Clear	44	
1	2024-07-15 04:07:23	21.353403	39.850382	High	0.85	Resting	Clear	39	
2	2024-06-06 08:27:11	21.252375	39.937753	High	0.94	Prayer	Clear	32	
3	2024-12-23 06:29:22	21.377740	39.813333	High	0.55	Prayer	Clear	41	
4	2024-08-19 18:00:22	21.360457	39.950974	Low	0.10	Sa'i	Rainy	44	
...	...	...	...	...	...	...	...	...	
9995	2024-01-01 23:57:15	21.209851	39.928508	Medium	0.50	Tawaf	Clear	38	
9996	2024-02-18 19:18:55	21.348037	39.854530	High	0.71	Walking	Clear	45	
9997	2024-04-05 04:01:17	21.330442	39.907906	High	0.75	Tawaf	Rainy	41	
9998	2024-04-22 03:49:37	21.209949	39.865282	Medium	1.65	Sa'i	Rainy	34	
9999	2024-11-29 18:52:24	21.275070	39.924463	Medium	0.10	Prayer	Rainy	43	

10000 rows × 34 columns

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
```

iii. Data Cleaning

iii.1. Eksplorasi Data Sederhana

```
#a. Mencari informasi jumlah baris dan kolom
df.shape
```

```
(10000, 34)
```

```
#b. Memunculkan tipe data
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 34 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Timestamp                       10000 non-null  object
 1   Location_Lat                    10000 non-null  float64
 2   Location_Long                   10000 non-null  float64
 3   Crowd_Density                   10000 non-null  object
 4   Movement_Speed                  10000 non-null  float64
 5   Activity_Type                   10000 non-null  object
 6   Weather_Conditions              10000 non-null  object
 7   Temperature                     10000 non-null  int64
 8   Sound_Level_dB                  10000 non-null  int64
 9   AR_System_Interaction           10000 non-null  object
 10  Fatigue_Level                   10000 non-null  object
 11  Stress_Level                    10000 non-null  object
 12  Queue_Time_minutes              10000 non-null  int64
```

```
 13  Health_Condition                10000 non-null  object
 14  Age_Group                       10000 non-null  object
 15  Nationality                     10000 non-null  object
 16  Transport_Mode                  10000 non-null  object
 17  Waiting_Time_for_Transport      10000 non-null  int64
 18  Security_Checkpoint_Wait_Time   10000 non-null  int64
 19  Emergency_Event                 10000 non-null  object
 20  Incident_Type                   10000 non-null  object
 21  Crowd_Morale                    10000 non-null  object
 22  Pilgrim_Experience              10000 non-null  object
 23  Interaction_Frequency           10000 non-null  int64
 24  Distance_Between_People_m       10000 non-null  float64
 25  Event_Type                      10000 non-null  object
 26  Time_Spent_at_Location_minutes  10000 non-null  int64
 27  AR_Navigation_Success           10000 non-null  object
 28  Satisfaction_Rating             10000 non-null  int64
 29  Perceived_Safety_Rating         10000 non-null  int64
 30  Month                           10000 non-null  int64
 31  Day                             10000 non-null  int64
 32  Hour                            10000 non-null  int64
 33  Is_Hajj_Season                  10000 non-null  int64
dtypes: float64(4), int64(13), object(17)
memory usage: 2.6+ MB
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
```

```
#c. Melihat statistik deskriptif
df.describe()
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
```

	Location_Lat	Location_Long	Movement_Speed	Temperature	Sound_Level_dB	Queue_Time_minutes	Waiting_Time_for_Transport
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	21.299628	39.898729	0.849895	37.515400	75.583400	31.456500	57.249500
std	0.057664	0.057236	0.442306	4.610736	11.197392	22.141502	32.738947
min	21.200078	39.800043	0.100000	30.000000	60.000000	0.000000	0.000000
25%	21.249454	39.849039	0.520000	33.000000	66.000000	13.000000	30.000000
50%	21.299998	39.899346	0.840000	38.000000	75.000000	30.000000	59.000000
75%	21.349764	39.947166	1.180000	42.000000	84.000000	48.000000	90.000000
max	21.399917	39.999994	1.800000	45.000000	100.000000	80.000000	120.000000

iii.2. Missing Value Check & Handling

```
#a. Mengecek jumlah missing values per kolom
print(df.isnull().sum())
```

```
Timestamp                       0
Location_Lat                    0
Location_Long                   0
Crowd_Density                   0
Movement_Speed                  0
Activity_Type                   0
Weather_Conditions              0
Temperature                     0
Sound_Level_dB                  0
AR_System_Interaction           0
Fatigue_Level                   0
Stress_Level                    0
Queue_Time_minutes              0
Health_Condition                0
Age_Group                       0
Nationality                     0
Transport_Mode                  0
Waiting_Time_for_Transport      0
Security_Checkpoint_Wait_Time   0
Emergency_Event                 0
Incident_Type                   0
Crowd_Morale                    0
Pilgrim_Experience              0
Interaction_Frequency           0
```

```
Distance_Between_People_m         0
Event_Type                        0
Time_Spent_at_Location_minutes    0
AR_Navigation_Success             0
Satisfaction_Rating               0
Perceived_Safety_Rating           0
Month                             0
Day                               0
Hour                              0
Is_Hajj_Season                    0
dtype: int64
```

iii.3. Duplicate Data Check & Handling

```
#a. Mengecek jumlah data duplikat
print(f'Jumlah duplikat: {df.duplicated().sum()}')
```

```
Jumlah duplikat: 0
```

iii.4. Inconsistency Data Check & Handling

```
#a. Melakukan pengecekan di daftar kategori unik pada kolom Incident_Type
print(df['Incident_Type'].unique())
```

```
['Theft' 'Security Breach' 'Unruly Behavior' 'Lost Item'
 'Medical Emergency']
```

```
#b. Melakukan pengecekan daftar kategori unik pada kolom Age_Group
print(df['Age_Group'].unique())
```

```
['Youth']
```

```
#c. Melakukan pengecekan daftar kategori unik pada kolom Activity_Type
print(df['Activity_Type'].unique())
```

```
['Tawaf' 'Resting' 'Prayer' 'Sa’i' 'Walking']
```

```
#d. Melakukan pengecekan daftar kategori unik pada kolom Weather_Conditions
print(df['Weather_Conditions'].unique())
```

```
['Clear' 'Rainy' 'Cloudy']
```

iii.5. Final Data Check

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 34 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Timestamp                       10000 non-null  object
 1   Location_Lat                    10000 non-null  float64
 2   Location_Long                   10000 non-null  float64
 3   Crowd_Density                   10000 non-null  object
 4   Movement_Speed                  10000 non-null  float64
 5   Activity_Type                   10000 non-null  object
 6   Weather_Conditions              10000 non-null  object
 7   Temperature                     10000 non-null  int64
 8   Sound_Level_dB                  10000 non-null  int64
 9   AR_System_Interaction           10000 non-null  object
 10  Fatigue_Level                   10000 non-null  object
 11  Stress_Level                    10000 non-null  object
 12  Queue_Time_minutes              10000 non-null  int64
 13  Health_Condition                10000 non-null  object
 14  Age_Group                       10000 non-null  object
 15  Nationality                     10000 non-null  object
 16  Transport_Mode                  10000 non-null  object
 17  Waiting_Time_for_Transport      10000 non-null  int64
 18  Security_Checkpoint_Wait_Time   10000 non-null  int64
 19  Emergency_Event                 10000 non-null  object
 20  Incident_Type                   10000 non-null  object
 21  Crowd_Morale                    10000 non-null  object
 22  Pilgrim_Experience              10000 non-null  object
 23  Interaction_Frequency           10000 non-null  int64
 24  Distance_Between_People_m       10000 non-null  float64
```

```
 25  Event_Type                      10000 non-null  object
 26  Time_Spent_at_Location_minutes  10000 non-null  int64
 27  AR_Navigation_Success           10000 non-null  object
 28  Satisfaction_Rating             10000 non-null  int64
 29  Perceived_Safety_Rating         10000 non-null  int64
 30  Month                           10000 non-null  int64
 31  Day                             10000 non-null  int64
 32  Hour                            10000 non-null  int64
 33  Is_Hajj_Season                  10000 non-null  int64
dtypes: float64(4), int64(13), object(17)
memory usage: 2.6+ MB
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
```

iv. Great Expectations

iv.1. Setup Data Context & Validator

```
#a. Membuat context otomatis di memori notebook
context = ge.get_context()
```

```
INFO:great_expectations.data_context.types.base:Created temporary directory '/tmp/tmpxqaiqmsk' for ephemeral docs site
```

```
#b. Membuat atau Mengambil Datasource
datasource_name = 'hajj_datasource' #Memberi nama unik pada datasource
try:
    datasource = context.data_sources.add_pandas(name=datasource_name) #Mencoba membuat datasource baru
except:
    datasource = context.data_sources.get(datasource_name) #Menggunakan nama datasource yang sudah ada jika sudah terdaftar

#c. Membuat Data Asset dari df
asset_name = 'hajj_asset' #memberi nama aset data
try:
    asset = datasource.add_dataframe_asset(name=asset_name) #mendaftarkan df sebagai aset baru
except:
    asset = datasource.get_asset(asset_name) #mengambil aset yang sudah ada jika sudah terdaftar
```

```
# d. Memberi nama untuk aturan validasi
expectation_suite_name = 'hajj_expectation_suite'

# e. Membuat Expectation Suite
try:
    # Membuat suite baru di dalam context
    suite = context.suites.add(ge.ExpectationSuite(name=expectation_suite_name))
except:
    # Mengambil suite yang sudah ada jika cell dijalankan ulang
    suite = context.suites.get(name=expectation_suite_name)

# f. Membuat validator
validator = context.get_validator(
    batch_request = asset.build_batch_request(options={"dataframe": df}), # untuk memasukkan dataframe
    expectation_suite_name = expectation_suite_name
)

# g. Mengecek Validator
# Menampilkan 5 baris pertama
validator.head()
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecat
  return datetime.utcnow().replace(tzinfo=utc)
```

Calculating Metrics: 100% 1/1 [00:00<00:00, 39.57it/s]

	Timestamp	Location_Lat	Location_Long	Crowd_Density	Movement_Speed	Activity_Type	Weather_Conditions	Temperature	Sound_L
0	2024-02-28 05:36:50	21.258846	39.983949	Medium	0.60	Tawaf	Clear	44	
1	2024-07-15 04:07:23	21.353403	39.850382	High	0.85	Resting	Clear	39	
2	2024-06-06 08:27:11	21.252375	39.937753	High	0.94	Prayer	Clear	32	
3	2024-12-23 06:29:22	21.377740	39.813333	High	0.55	Prayer	Clear	41	
4	2024-08-19 18:00:22	21.360457	39.950974	Low	0.10	Sa'i	Rainy	44	

5 rows × 34 columns

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecat
  return datetime.utcnow().replace(tzinfo=utc)
```

iv.2. Great Expectations

```
#1. To be Unique
#a. Menambahkan kolom ID unik baru berdasarkan urutan baris agar validasi sukses
df['id_unit'] = range(1, len(df)+1)

#b. Memperbarui validator agar menyertakan kolom baru
validator = context.get_validator(
    batch_request = asset.build_batch_request(options={"dataframe": df}), # untuk memuat ulang dataframe
    expectation_suite_name = expectation_suite_name
)

#c. Menjalankan ekspektasi pertama
validator.expect_column_values_to_be_unique("id_unit")
```

```
/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1633: UserWarning: `result_format` config
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
Calculating Metrics: 100%                                    8/8 [00:00<00:00, 229.74it/s]
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
{
  "success": true,
  "expectation_config": {
    "type": "expect_column_values_to_be_unique",
    "kwargs": {
      "batch_id": "hajj_datasource-hajj_asset",
      "column": "id_unit"
    },
    "meta": {},
    "severity": "critical"
  },
  "result": {
    "element_count": 10000,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "partial_unexpected_list": [],
    "missing_count": 0,
    "missing_percent": 0.0,
    "unexpected_percent_total": 0.0,
    "unexpected_percent_nonmissing": 0.0
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```

```
#2. To be Between (memastikan kecepatan jamaah berada di rantang normal 0 hingga 5 m/s agar data tetap logis
validator.expect_column_values_to_be_between("Movement_Speed", min_value=0, max_value=5)
```

```
/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1633: UserWarning: `result_format` config
  warnings.warn(
Calculating Metrics: 100%                                    8/8 [00:00<00:00, 162.11it/s]
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
{
  "success": true,
  "expectation_config": {
    "type": "expect_column_values_to_be_between",
    "kwargs": {
      "batch_id": "hajj_datasource-hajj_asset",
      "column": "Movement_Speed",
      "min_value": 0.0,
      "max_value": 5.0
    },
    "meta": {},
    "severity": "critical"
  },
  "result": {
    "element_count": 10000,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "partial_unexpected_list": [],
    "missing_count": 0,
    "missing_percent": 0.0,
    "unexpected_percent_total": 0.0,
    "unexpected_percent_nonmissing": 0.0
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```

```
#3. To be in Set (Memastikan kategori kepadatan hanya berisi nilaii standar (Low, Medium, High)).
validator.expect_column_values_to_be_in_set("Crowd_Density", ["Low", "Medium", "High"])
```

```
/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1633: UserWarning: `result_format` config
  warnings.warn(
Calculating Metrics: 100%                                                    8/8 [00:00<00:00, 117.33it/s]
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
{
  "success": true,
  "expectation_config": {
    "type": "expect_column_values_to_be_in_set",
    "kwargs": {
      "batch_id": "hajj_datasource-hajj_asset",
      "column": "Crowd_Density",
      "value_set": [
        "Low",
        "Medium",
        "High"
      ]
    },
    "meta": {},
    "severity": "critical"
  },
  "result": {
    "element_count": 10000,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "partial_unexpected_list": [],
    "missing_count": 0,
    "missing_percent": 0.0,
    "unexpected_percent_total": 0.0,
    "unexpected_percent_nonmissing": 0.0
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```

```
#4. To be in Type List (Memastikan kolom Latitude bertipe float)
validator.expect_column_values_to_be_of_type("Location_Lat", "float")
```

```
/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1633: UserWarning: `result_format` config
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
Calculating Metrics: 100%                                                    1/1 [00:00<00:00, 40.13it/s]
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
{
  "success": true,
  "expectation_config": {
    "type": "expect_column_values_to_be_of_type",
    "kwargs": {
      "batch_id": "hajj_datasource-hajj_asset",
      "column": "Location_Lat",
      "type_": "float"
    },
    "meta": {},
    "severity": "critical"
  },
  "result": {
    "observed_value": "float64"
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```

```
#5. Data Not Null (Memastikan data aktivitas jamaah tidak ada yang kosong)
validator.expect_column_values_to_not_be_null("Activity_Type")
```

```
/usr/local/lib/python3.12/dist-packages/great_expectations/expectations/expectation.py:1633: UserWarning: `result_format` config
  warnings.warn(
```

Calculating Metrics: 100% 6/6 [00:00<00:00, 189.17it/s]

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is depreca
  return datetime.utcnow().replace(tzinfo=utc)
{
  "success": true,
  "expectation_config": {
    "type": "expect_column_values_to_not_be_null",
    "kwargs": {
      "batch_id": "hajj_datasource-hajj_asset",
      "column": "Activity_Type"
    },
    "meta": {},
    "severity": "critical"
  },
  "result": {
    "element_count": 10000,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "partial_unexpected_list": []
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
```